

Akhil Bodahanapati

University of Pittsburgh, Computer Science, Expected December 2027

Easton & Easton College Scholarship

What are the top three skills you are working to cultivate that will help you achieve your career goals?

I want to be a software engineer who builds things that hold up when real people use them. I've had enough internships and side projects by now to know that writing code that works on my laptop is the easy part. The skills I'm working on are the ones that come after that.

The first is thinking about what happens when things go wrong.

When I started programming, I wrote code for the happy path. The user types the right input, the network is up, the database answers. Then I built InterviewSense, an AI interview practice platform, and real people started using it. Today it has more than 25,000 users. Real users find every edge case you didn't think about, and some you couldn't have. One early problem: we charged usage credits before calling an AI model, and if the model request failed, the user lost the credit and got nothing. Nobody complained loudly, but I noticed the support emails. I rebuilt the billing flow so failed requests automatically gave the credit back. It was a small fix, but it was the first time I thought about failure as something to design for instead of something to apologize for.

I got a bigger version of that lesson this summer at Nutanix, where I interned on the control plane for their cloud infrastructure. I worked on how partial updates were applied to customer clusters. The old logic could, in certain situations, overwrite settings a customer never meant to change. You don't notice a bug like that until it hits someone's production system. I redesigned the update path to preserve existing state, and added checks for conflicting writes when two requests hit the same resource at once. I'm still learning how to anticipate those cases before they happen. My instinct now is to ask "what's the worst thing this could do" before I ask "does it work."

The second skill is making things faster without making them fragile.

At Pitt's Natural Language Processing lab, researchers ran experiments over roughly a million documents. Each run took about four minutes, which meant people waited around a lot and ran

fewer experiments than they wanted to. I dug into it and found that most of the time was spent in slow database queries and in work that could have been done in parallel. I rewrote the PostgreSQL queries and added Redis-backed job queues, and turnaround dropped to about seventy-five seconds.

What I took from that wasn't really about databases. It was that performance work is mostly patience. You measure, you find the one thing that matters, and you leave everything else alone. I used to want to rewrite whole systems. Now I'm more interested in finding the one slow line. I'm practicing this in my coursework too, especially in Database Management Systems this semester, where I can see why the tricks I used at the lab worked.

The third skill is communicating clearly, and I'll be honest that this is the one I'm furthest behind on.

I'm better at talking to computers than to people. In my sophomore spring, I withdrew from a professional writing course because I was overloaded, and I knew at the time that I was dropping the class I probably needed most. I'm retaking a professional communication course this semester and taking it seriously. At work I've started to notice how much good engineering depends on writing. A design doc that's clear saves a week of meetings. A pull request description that explains the "why" saves a reviewer an hour. At Federated Hermes I worked directly with portfolio managers to replace manual workflows with backend services, and the hard part was never the Python. It was understanding what they actually needed and explaining what I could build in words that made sense to someone who doesn't write code.

I've also seen what happens when communication goes well. At the Nutanix intern hackathon, my team built a Kubernetes scheduler that automatically parked idle GPUs, and we won for the US and EMEA region. The project was good, but I think we won because we explained it in two minutes in a way that made judges care about GPU waste. I want to be able to do that on purpose, not by accident.

These three skills fit together for me. I want to build systems that survive mistakes, that are fast enough people actually use them, and that other people can understand and trust. I'm graduating from the University of Pittsburgh with a computer science degree in December 2027. After that I want to work on infrastructure and backend systems, the kind of software that most people never see but that everything depends on.

My family has dealt with health problems and financial strain for a long time, including a move across the country from California to Pennsylvania. Those years taught me to work hard without assuming things would go smoothly. This scholarship would help cover my last semesters so I can spend more time getting better at these skills and less time worrying about how to pay for them.